

Segurando sistemas em Docker numa VM (Proxmox)

Tutorial geral e reutilizável — aplique este roteiro a qualquer sistema novo que você suba na VM, não só a um específico.

Roteiro

1. Princípio geral: por que fazer isso
2. Padrão de portas por container
3. Docker Compose seguro (checklist + exemplo)
4. Nginx como porta de entrada única
5. TLS com Let's Encrypt
6. Autenticação (basic auth quando o sistema não tem login próprio)
7. Firewall da VM
8. SSH
9. Segredos e credenciais
10. Rotina de manutenção
11. Checklist final por sistema novo

1. Princípio geral

Numa VM com vários sistemas em Docker, cada porta publicada é uma porta de entrada em potencial. A estratégia central deste tutorial é:

- **Nada fala com o mundo exceto o Nginx.** Todo container escuta só em `127.0.0.1`; o Nginx é o único processo que aceita conexões de fora (portas 80/443).
- **Feche tudo por padrão no firewall** e libere só o que precisa.
- **Documente as portas em uso** pra nunca colidir nem perder o controle.

2. Padrão de portas por container

Reserve uma faixa "sua" para as portas internas dos seus sistemas — por exemplo `3000-3999` para apps e `8000-8099` para outros serviços internos. Evite:

- Portas privilegiadas (< 1024, exigem root)

- Portas famosas de serviços (3306 MySQL, 5432 Postgres, 6379 Redis, 3389 RDP, 5900 VNC, 27017 MongoDB) — scanners testam essas primeiro

Mantenha uma tabela simples, atualizada a cada sistema novo:

Porta interna	Sistema	Container	Domínio no Nginx
8010	Pingador	pingador	pingador.seudominio.com
3020	App "4m"	4m_app	checklist.seudominio.com
27017	MongoDB (4m)	4m_mongodb	<i>não exposto</i>

A porta interna nunca aparece pra quem acessa de fora — o usuário só vê o domínio (`https://sistema.seudominio.com`). A porta só importa internamente, pro Nginx e pro `docker-compose.yml` saberem para onde mandar a requisição.

3. Docker Compose seguro

Checklist ao escrever/revisar um `docker-compose.yml`

- ☐ Toda porta que só o Nginx da mesma VM precisa acessar usa prefixo `127.0.0.1:`
- ☐ Bancos de dados/caches que só outro container usa não publicam porta nenhuma — só entram na mesma `network` e são acessados pelo nome do serviço (DNS interno do Docker)
- ☐ Sem senha default fraca (nada de `changeme` , `admin` , `123456`)
- ☐ `.env` com as credenciais reais nunca é versionado no git
- ☐ `restart: unless-stopped` em tudo que deve sobreviver a reboot

Exemplo — app + banco (sem expor nada além do necessário)

```
services:
  app:
    build: .
    container_name: meusistema_app
    restart: unless-stopped
    env_file: .env
    environment:
      PORT: 3020
      MONGODB_URI: mongodb://${DB_USER:?defina DB_USER}:${DB_PASSWORD:?defina DB_PASSWORD}
    ports:
      - "127.0.0.1:3020:3020"          # só o Nginx local acessa
    depends_on:
      db:
        condition: service_healthy
    networks:
      - meusistema_net

  db:
    image: mongo:4.4
    container_name: meusistema_db
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${DB_USER:?defina DB_USER}
      MONGO_INITDB_ROOT_PASSWORD: ${DB_PASSWORD:?defina DB_PASSWORD}
    volumes:
      - db_data:/data/db
    # sem "ports:" -- só o container "app" precisa falar com o banco,
    # e ele já enxerga via rede interna (nome do serviço = hostname)
    networks:
      - meusistema_net

networks:
  meusistema_net:
    driver: bridge

volumes:
  db_data:
```

4. Nginx como porta de entrada única

Um bloco `server{}` por sistema, todos usando o mesmo padrão. Troque `sistema.seudominio.com` e a porta pelo do seu caso:

```

server {
    listen 80;
    server_name sistema.seudominio.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name sistema.seudominio.com;

    ssl_certificate      /etc/letsencrypt/live/sistema.seudominio.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/sistema.seudominio.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:3020;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

Salvar, habilitar e recarregar:

```

sudo nano /etc/nginx/sites-available/sistema
sudo ln -s /etc/nginx/sites-available/sistema /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx

```

5. TLS com Let's Encrypt

Com o DNS do subdomínio já apontado para o IP da VM:

```

sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d sistema.seudominio.com

```

O certbot edita o bloco `server{}` automaticamente (preenche `ssl_certificate` / `ssl_certificate_key`) e configura a renovação automática via `systemd timer` . Confirme com:

```

sudo systemctl status certbot.timer

```

6. Autenticação (quando o sistema não tem login próprio)

Sistemas sem tela de login (MVPs, dashboards internos, ferramentas de uso próprio) ficam abertos pra qualquer um que descubra a URL. Se o sistema já tem autenticação de verdade, pule esta etapa.

```
sudo apt install -y apache2-utils
sudo htpasswd -c /etc/nginx/.htpasswd-sistema usuario
```

Dentro do bloco `server{}` (HTTPS), antes do `proxy_pass` :

```
location / {
    auth_basic "Restrito";
    auth_basic_user_file /etc/nginx/.htpasswd-sistema;

    proxy_pass http://127.0.0.1:3020;
    ...
}
```

7. Firewall da VM

Fechado por padrão, libere só o essencial:

```
sudo ufw default deny incoming
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow from <sua-rede-de-gerencia>/24 to any port 22 proto tcp
sudo ufw enable
sudo ufw status verbose
```

Repita a mesma lógica no firewall do Proxmox (nível de VM/datacenter) como camada redundante — se um mecanismo falhar, o outro segura.

8. SSH

- Login só por chave: em `/etc/ssh/sshd_config`, defina `PasswordAuthentication no` e reinicie (`sudo systemctl restart sshd`)
- Instale e habilite o `fail2ban` pra banir tentativas de força bruta

- Restrinja a porta 22 por IP de origem no firewall (passo 7)

9. Segredos e credenciais

- Um `.env` por sistema, fora do controle de versão (`.gitignore`)
- Senhas geradas (`openssl rand -base64 24`), nunca defaults tipo `admin` / `changeme`
- No compose, prefira `${VAR:?defina VAR no .env}` a `${VAR:-changeme}` — assim o serviço recusa subir com valor fraco/ausente em vez de aceitar um default inseguro silenciosamente

10. Rotina de manutenção

- `unattended-upgrades` habilitado no SO da VM (patches de segurança automáticos)
- Rebuild periódico das imagens Docker para pegar patches da base (`docker compose pull && docker compose up -d --build`)
- Backup regular da VM (snapshot Proxmox / `vzdump`), especialmente antes de mudanças estruturais
- Checar `/var/log/nginx/access.log` e `error.log` de vez em quando — primeiro sinal de scan ou tentativa de exploit

11. Checklist final — a cada sistema novo que você subir

- ☐ Porta interna escolhida sem colidir com a tabela de portas em uso
- ☐ `ports:` do compose com prefixo `127.0.0.1:` (ou sem `ports:` nenhuma, se só outro container acessa)
- ☐ Credenciais fortes no `.env` , não versionado
- ☐ Subdomínio criado no DNS apontando pro IP da VM
- ☐ Bloco Nginx criado, testado com `nginx -t` e recarregado
- ☐ Certificado TLS emitido com certbot
- ☐ Basic auth adicionado se o sistema não tem login próprio
- ☐ Testado de fora: porta interna não responde diretamente (`curl http://ip-da-vm:porta` deve falhar), só o domínio HTTPS funciona
- ☐ Tabela de portas atualizada com a entrada nova

Tutorial gerado a partir da conversa sobre o deploy do sistema Pingador — aplica-se a qualquer sistema Docker novo na mesma VM.